

ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC BÁCH KHOA
KHOA KHOA HỌC VÀ KỸ THUẬT MÁY TÍNH



Xử lý tín hiệu số (CO2036)

Báo cáo Lab 1 - 2

Lớp L03 - HK252

Giảng viên hướng dẫn: Phạm Công Thái

Sinh viên thực hiện: Nguyễn Ngọc Quỳnh Nhi

Mã số sinh viên: 2212438

Thành phố Hồ Chí Minh, tháng 03 năm 2026

Mục lục

1	Giới thiệu	2
2	Mục tiêu	2
3	Phương pháp thực hiện	3
4	Nội dung thực hành	4
4.1	Bài thực hành 1	4
4.1.1	Exercise 1.1	4
4.1.2	Exercise 1.2	6
4.2	Bài thực hành 2	10
4.2.1	Exercise 1	10
4.2.2	Exercise 2	11
4.2.3	Exercise 3	12
4.2.4	Exercise 4	13
4.2.5	Exercise 5	14
4.2.6	Exercise 6	16
4.2.7	Exercise 7	19
4.2.8	Exercise 8	21
5	Bàn luận	24
5.1	Tổng kết kết quả thực hiện	24
5.2	Hạn chế và những điểm chưa hoàn thiện	24
5.3	Định hướng phát triển kỹ năng	25
6	Tài liệu tham khảo	25

Tổng quan

Báo cáo này tổng hợp kết quả thực hiện các bài tập thực hành thuộc Lab 1 và Lab 2 trong khuôn khổ môn học **Xử lý tín hiệu số (CO2036)**, học kỳ 2 năm học 2025-2026. Nội dung báo cáo tập trung vào việc ứng dụng phần mềm Scilab để giải quyết các bài toán cơ bản về xử lý tín hiệu rời rạc, từ việc làm quen với môi trường lập trình đến các phép biến đổi tín hiệu phức tạp.

1 Giới thiệu

Trong kỷ nguyên công nghệ số, Xử lý tín hiệu số (DSP) đóng vai trò nền tảng trong nhiều lĩnh vực kỹ thuật. Việc nắm vững các công cụ mô phỏng như Scilab không chỉ giúp hiện thực hóa các lý thuyết toán học mà còn giúp sinh viên quan sát trực quan sự thay đổi của tín hiệu trong miền thời gian và miền tần số. Bài thực hành này là bước đệm quan trọng giúp sinh viên chuyển đổi tư duy từ tín hiệu tương tự (analog) sang tín hiệu rời rạc (discrete), vốn là cốt lõi của các hệ thống xử lý hiện đại.

2 Mục tiêu

Việc thực hiện các bài Lab 1 và Lab 2 hướng tới các mục tiêu cụ thể sau:

- **Về kỹ năng phần mềm:** Làm quen với cú pháp, các phép toán vector/ma trận và khả năng đồ họa mạnh mẽ của Scilab.
- **Về kiến thức chuyên môn:**
 - Hiểu rõ quá trình lấy mẫu và lượng tử hóa tín hiệu thông qua thực nghiệm mô phỏng.
 - Xây dựng và biểu diễn thành thạo các tín hiệu cơ bản: xung đơn vị $\delta(n)$, bước nhảy đơn vị $u(n)$, và dốc đơn vị $u_r(n)$.
 - Làm chủ các phép biến đổi tín hiệu: dịch thời gian, đảo thời gian, cộng/nhân tín hiệu và phân tách thành phần chẵn/lẻ.

3 Phương pháp thực hiện

Để hoàn thành nội dung thực hành, các phương pháp sau đã được áp dụng:

1. **Phân tích lý thuyết:** Nghiên cứu biểu thức toán học của các tín hiệu và phép biến đổi trước khi lập trình để xác định kết quả mong đợi (miền thời gian, biên độ).
2. **Lập trình mô phỏng:** Sử dụng công cụ SciNotes của phần mềm Scilab để viết các kịch bản lệnh (scripts). Ứng dụng tư duy *vectorization* để tối ưu hóa việc tính toán thay vì sử dụng vòng lặp thủ công.
3. **Trực quan hóa dữ liệu:** Sử dụng các hàm đồ họa chuyên dụng như `plot2d3`, `subplot` để vẽ đồ thị tín hiệu rời rạc (stem plot), giúp so sánh và đánh giá sự khác biệt giữa các tín hiệu gốc và tín hiệu đã qua biến đổi.

4 Nội dung thực hành

4.1 Bài thực hành 1

4.1.1 Exercise 1.1

Sử dụng các toán tử vector và ma trận để thực hiện các yêu cầu sau:

a. Tạo 1 vector có dạng $(x_1 + 1, x_2 + 1, x_3 + 1, x_4 + 1)$ với x_1, x_2, x_3, x_4 là các phần tử của vector $x = 1 : 4$.

b. Tạo 1 vector có dạng $(x_1 y_1, x_2 y_2, x_3 y_3, x_4 y_4)$ với x_1, x_2, x_3, x_4 là các phần tử của vector $x = 1 : 4$ và y_1, y_2, y_3, y_4 là các phần tử của vector $y = 5 : 8$.

c. Tạo 1 vector có dạng $(\sin(x_1), \sin(x_2), \dots, \sin(x_{10}))$ với x là 1 vector gồm 10 giá trị tuyến tính trong khoảng $[0, \pi]$.

Bài làm

a. Tạo 1 vector có dạng $(x_1 + 1, x_2 + 1, x_3 + 1, x_4 + 1)$ với x_1, x_2, x_3, x_4 là các phần tử của vector $x = 1 : 4$.

Để thực hiện được yêu cầu của câu a, đầu tiên ta định nghĩa vector $x = 1 : 4$ bằng cú pháp `x = 1:4;`. Sau đó dùng toán tử cộng (+) để thực hiện cộng 1 vào từng phần tử của vector x , rồi gán kết quả sau khi cộng vào biến `v1`, với cú pháp lệnh là `v1 = x + 1;`. Cuối cùng, ta thực hiện in kết quả ra màn hình `disp(v1);`.

```
1 // Task 1: Vector plus constant
2 x = 1:4;
3 v1 = x + 1;
4 disp(v1);
```

Listing 1: Code Scilab cho Exercise 1.1a

```
Scilab 2026.0.1 Console

--> // Task 1: Vector plus constant

--> x = 1:4;

--> disp(x);

    1.    2.    3.    4.

--> v1 = x + 1;

--> disp(v1);

    2.    3.    4.    5.
```

Hình 1: Kết quả đoạn code SciLab cho Exercise 1.1a.

b. Tạo 1 vector có dạng $(x_1y_1, x_2y_2, x_3y_3, x_4y_4)$ với x_1, x_2, x_3, x_4 là các phần tử của vector $x = 1 : 4$ và y_1, y_2, y_3, y_4 là các phần tử của vector $y = 5 : 8$.

Để thực hiện được yêu cầu của câu b, đầu tiên ta định nghĩa 2 vector $x = 1 : 4$ và $y = 5 : 8$ lần lượt bằng cú pháp lệnh `x = 1:4;` và `y = 5:8;`. Sau đó, ta dùng toán tử `.*` để thực hiện nhân từng phần tử tương ứng của 2 vector với nhau, lưu kết quả vào biến `v2` rồi hiển thị ra màn hình.

```
1 // Task 2: Element-wise multiplication
2 x = 1:4;
3 disp(x);
4 y = 5:8;
5 disp(y);
6 v2 = x .* y;    // Nhân x1*y1, x2*y2...
7 disp(v2);
```

Listing 2: Code Scilab cho Exercise 1.1b

```
--> // Task 2: Element-wise multiplication
--> x = 1:4;
--> disp(x);
    1.    2.    3.    4.
--> y = 5:8;
--> disp(y);
    5.    6.    7.    8.
--> v2 = x .* y;    // Nhân x1*y1, x2*y2...
--> disp(v2);
    5.   12.   21.   32.
```

Hình 2: Kết quả đoạn code SciLab cho Exercise 1.1b

c. Tạo 1 vector có dạng $(\sin(x_1), \sin(x_2), \dots, \sin(x_{10}))$ với x là 1 vector gồm 10 giá trị tuyến tính trong khoảng $[0, \pi]$.

Để thực hiện được yêu cầu của câu c, ta cần sử dụng hàm `linspace(a, b, n)` với a, b là 2 số thực và n là 1 số nguyên, để định nghĩa 1 vector gồm n giá trị trong khoảng từ a đến b . Đầu tiên, ta tạo 1 vector x gồm 10 giá trị trong khoảng từ $[0, \pi]$ với lệnh `x = linspace(0, %pi, 10);`. Sau đó tính $\sin(x)$ và gán kết quả này vào biến `v3`. Cuối cùng, in kết quả `v3` ra màn hình `disp(v3);`.

```
1 // Task 3: Sine of linearly spaced vector
2 x = linspace(0, %pi, 10);
3 v3 = sin(x);
4 disp(v3);
```

Listing 3: Code Scilab cho Exercise 1.1c

```
--> // Task 3: Sine of linearly spaced vector
--> x = linspace(0, %pi, 10);
--> v3 = sin(x);
--> disp(v3);
      column 1 to 7
      0.   0.3420201  0.6427876  0.8660254  0.9848078  0.9848078  0.8660254
      column 8 to 10
      0.6427876  0.3420201  1.225D-16
--> |
```

Hình 3: Kết quả đoạn code SciLab cho Exercise 1.1c

4.1.2 Exercise 1.2

Cho tín hiệu tương tự (Analog signal) $x_a(t) = 3\sin(100\pi t)$.

- Sử dụng SciLab vẽ tín hiệu $x_a(t)$ trong 5 chu kỳ.
- Xác định tín hiệu rời rạc thời gian (discrete-time signal) $x(n)$ của tín hiệu $x_a(t)$ được lấy mẫu với tần số lấy mẫu là $F_s = 300$ mẫu / giây.
- Xác định tín hiệu rời rạc thời gian $x(n)$ và xác định tính tuần hoàn (periodic property) của $x(n)$. Nếu $x(n)$ là tín hiệu tuần hoàn (periodic signal), xác định tần số và chu kỳ của $x(n)$. Sau đó, sử dụng SciLab để vẽ $x(n)$ trong 5 chu kỳ.
- Xác định tín hiệu lượng tử (quantized signal) $x_q(n)$ nếu $\Delta = 0.1$, sử dụng phương pháp cắt bỏ (truncated method). Sau đó, vẽ x_q trong 5 chu kỳ.

Lưu ý: tất cả hình ảnh bao gồm $x_a(t)$, $x(n)$, và $x_q(n)$ phải được trình bày trong cùng 1 cửa sổ.

a. Sử dụng SciLab vẽ tín hiệu $x_a(t)$ trong 5 chu kỳ.

Từ tín hiệu $x_a(t) = 3\sin(100\pi t)$ ta có:

$$2\pi F = 100\pi \Rightarrow F = 50(\text{Hz}) \Rightarrow T = 0.02(\text{s}) \Rightarrow 5T = 0.1(\text{s})$$

Đề bài yêu cầu vẽ tín hiệu $x_a(t)$ trong 5 chu kỳ tức là ta sẽ vẽ tín hiệu $x_a(t)$ trong khoảng thời gian từ 0 giây đến 0.1 giây.

b. Xác định **tín hiệu rời rạc thời gian (discrete-time signal)** $x(n)$ của tín hiệu $x_a(t)$ được lấy mẫu với tần số lấy mẫu là $F_s = 300$ mẫu / giây.

Theo định lý lấy mẫu $F_s > 2F_{max}$, ta có:

$$F_s = 300(\text{Hz}) > 2F_{max} = 2 \times 50 = 100(\text{Hz})$$

Vậy $F_s = 300$ (mẫu/s) thỏa mãn định lý lấy mẫu.

Xác định $x(n)$: Thay $t = \frac{n}{F_s} = \frac{n}{300}$ ta được:

$$x(n) = 3\sin(100\pi t \frac{n}{300}) = 3\sin(\frac{\pi}{3}n)$$

Vậy $x(n) = 3\sin(\frac{\pi}{3}n)$.

c. Xác định tín hiệu rời rạc thời gian $x(n)$ và xác định **tính tuần hoàn (periodic property)** của $x(n)$. Nếu $x(n)$ là **tín hiệu tuần hoàn (periodic signal)**, xác định tần số và chu kỳ của $x(n)$. Sau đó, sử dụng SciLab để vẽ $x(n)$ trong 5 chu kỳ.

Để xác định tín hiệu $x(n)$ có tính tuần hoàn hay không, ta thực hiện tìm chu kỳ của tín hiệu. Nếu tín hiệu $x(n)$ có chu kỳ thì $x(n)$ là tín hiệu tuần hoàn.

Từ biểu thức tín hiệu $x(n) = 3\sin(\frac{\pi}{3}n)$, suy ra vận tốc góc của tín hiệu là $\omega = \frac{\pi}{3}$.

Áp dụng công thức tính chu kỳ của tín hiệu rời rạc:

$$N = \frac{2\pi k}{\omega} = \frac{2\pi k}{\frac{\pi}{3}} = 6k \Rightarrow N_0 = 6 \text{ với } k = 1$$

$\Rightarrow x(n)$ là tín hiệu tuần hoàn với chu kỳ cơ bản là $N_0 = 6$ (mẫu/vòng) và tần số là $f = \frac{1}{N_0} = \frac{1}{6}$ (vòng/mẫu).

Đề bài yêu cầu vẽ tín hiệu $x(n)$ trong 5 chu kỳ, tức là ta sẽ vẽ các mẫu tín hiệu $x(n)$ khi $n = 0$ cho đến khi $n = 30$.

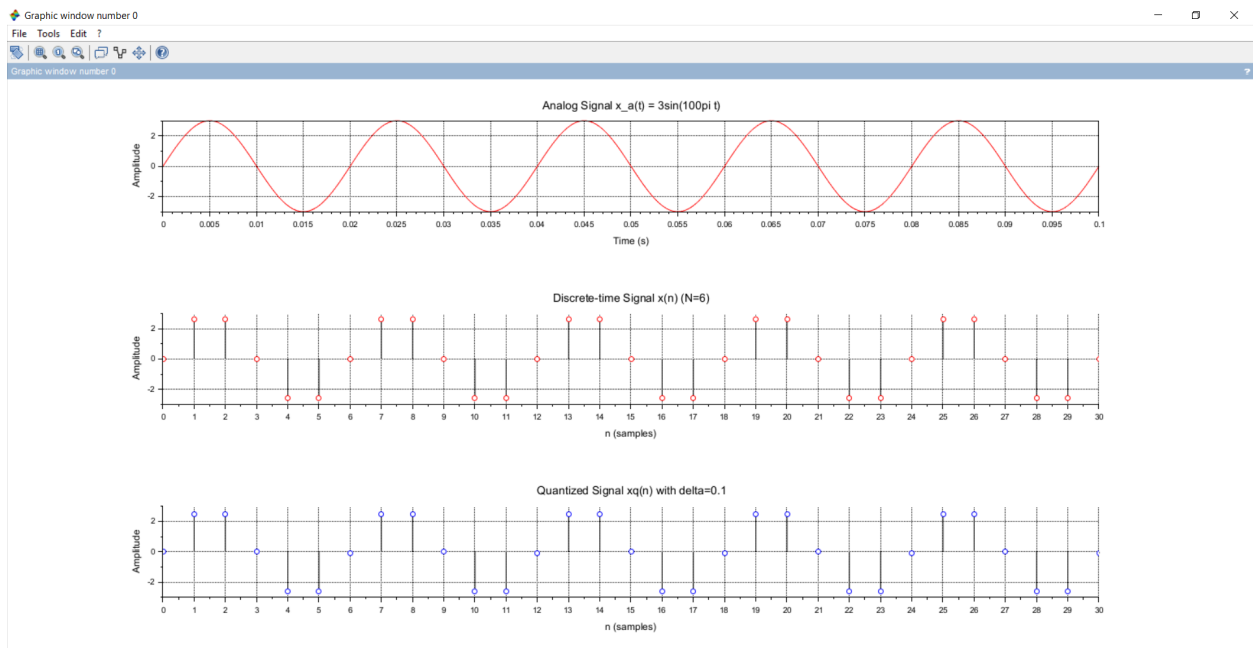
d. Xác định **tín hiệu lượng tử (quantized signal)** $x_q(n)$ nếu $\Delta = 0.1$, sử dụng **phương pháp cắt bỏ (truncated method)**. Sau đó, vẽ x_q trong 5 chu kỳ.

Áp dụng công thức $x_q(n) = \left\lfloor \frac{x(n)}{\Delta} \right\rfloor \Delta$ để xác định tín hiệu lượng tử $x_q(n)$ theo **phương pháp cắt bỏ (truncated method)**.

Tương tự với câu d, ta sẽ vẽ các mẫu tín hiệu $x_q(n)$ khi $n = 0$ cho đến khi $n = 30$.


```
1 clear; clc;
2 clf();
3
4 //----- Analog signal x_a(t) -----
5 t = linspace(0, 0.1, 1000);
6 x_a = 3 * sin(100 * %pi * t);
7
8 subplot(3, 1, 1);
9 plot(t, x_a, "red");
10 xtitle("Analog Signal x_a(t) = 3sin(100pi t)", "Time (s)", "Amplitude");
11 xgrid();
12
13 //----- Discrete-time signal x(n) -----
14 n = 0 : 30;
15 x_n = 3 * sin(%pi * n / 3);
16
17 subplot(3, 1, 2);
18 plot2d3(n, x_n);
19 plot(n, x_n, "ro");
20 xtitle("Discrete-time Signal x(n) (N=6)", "n (samples)", "Amplitude");
21 xgrid();
22
23 //----- Quantitized signal x_q(n) -----
24 delta = 0.1;
25 x_q = delta * floor(x_n / delta);
26
27 subplot(3, 1, 3);
28 plot2d3(n, x_q);
29 plot(n, x_q, "bo");
30 xtitle("Quantitized Signal xq(n) with delta=0.1", "n (samples)", "Amplitude");
31 xgrid();
```

Listing 4: Code Scilab cho Exercise 1.2



Hình 4: Kết quả Exercise 1.2.

4.2 Bài thực hành 2

4.2.1 Exercise 1

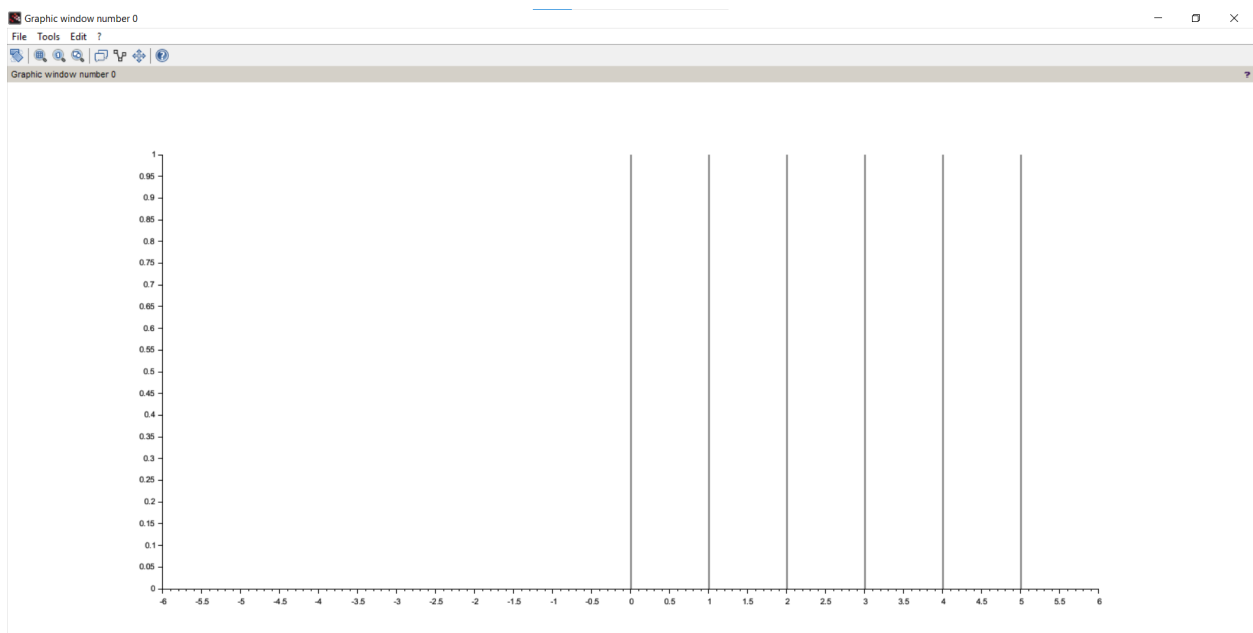
Hàm	Chức năng (Functionality)	Cách sử dụng (How to use it)
<code>plot2d3(...)</code>	Vẽ đồ thị 2D dưới dạng các thanh dọc (stem plot). Đây là công cụ quan trọng nhất để biểu diễn trực quan các tín hiệu rời rạc $x[n]$ trong miền thời gian.	<code>plot2d3(n, x);</code> Trong đó n là trục thời gian rời rạc (vector chỉ số mẫu) và x là giá trị biên độ của tín hiệu tại các mẫu đó.
<code>min(...)</code>	Tìm giá trị nhỏ nhất của các phần tử trong một vector hoặc ma trận.	<code>m = min(x);</code>
<code>max(...)</code>	Tìm giá trị lớn nhất của các phần tử trong một vector hoặc ma trận.	<code>M = max(x);</code>
<code>subplot(...)</code>	Chia cửa sổ đồ họa thành một lưới các ô nhỏ để vẽ nhiều đồ thị cùng lúc.	<code>subplot(m, n, p);</code> Chia cửa sổ thành lưới m hàng, n cột và chọn ô thứ p để vẽ.
<code>title(...)</code>	Thêm tiêu đề văn bản vào phía trên cùng của đồ thị hiện tại.	<code>title("Tên đồ thị");</code>
<code>xlabel(...)</code>	Thêm nhãn (chú thích) cho trục hoành (trục X).	<code>xlabel("Thời gian (s)");</code>
<code>ylabel(...)</code>	Thêm nhãn (chú thích) cho trục tung (trục Y).	<code>ylabel("Biên độ");</code>
<code>bool2s(...)</code>	Hàm chuyển đổi kiểu dữ liệu logic (%t cho True, %f cho False) thành kiểu số nguyên tương ứng (1 và 0).	<code>y = bool2s(x > 0);</code> Nếu biểu thức đúng thì trả về 1, sai trả về 0. Rất hữu ích khi tạo hàm bước nhảy đơn vị $u[n]$.
<code>deff(...)</code>	Định nghĩa một hàm mới một cách nhanh chóng bằng cách sử dụng các chuỗi ký tự.	<code>deff("y = f(x)", "y = x^2 + 1");</code> Tạo nhanh hàm số $f(x) = x^2 + 1$ để tính toán tức thời.

Bảng 1: Danh sách và giải thích chi tiết các hàm Scilab.

4.2.2 Exercise 2

Thực thi các đoạn mã sau trên Scilab và trình bày sự hiểu biết của bạn sau khi quan sát kết quả đầu ra.

```
1 scilab:> n = -5:5;  
2 scilab:> msignal = bool2s (n >= 0);  
3 scilab:> plot2d3(n, msignal)  
4
```



Hình 5: Kết quả đoạn code Scilab của Exercise 2.

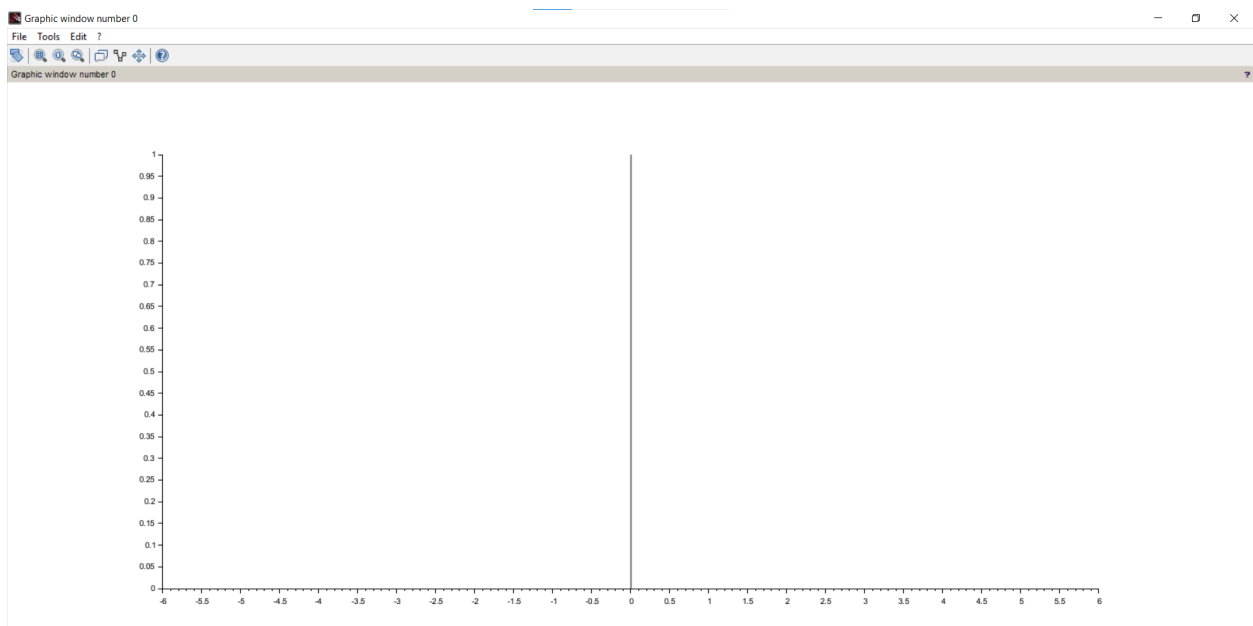
Giải thích kết quả Exercise 2:

- Thiết lập miền thời gian rời rạc n trong khoảng $[-5, 5]$.
- Tạo dãy **tín hiệu bước đơn vị (unit step signal)** $u[n]$ thông qua điều kiện $n \geq 0$. Hàm `bool2s` giúp chuyển đổi các giá trị logic thành biên độ 0 hoặc 1.
- Sử dụng hàm `plot2d3` để biểu diễn trực quan tín hiệu dưới dạng các mẫu rời rạc.

4.2.3 Exercise 3

Thực thi các đoạn mã sau trên Scilab và trình bày sự hiểu biết của bạn sau khi quan sát kết quả đầu ra.

```
1 scilab:> n = -5:5;
2 scilab:> msignal = bool2s (n == 0);
3 scilab:> plot2d3(n, msignal)
4
```



Hình 6: Kết quả đoạn code Scilab của Exercise 3.

Giải thích kết quả Exercise 3:

- Thiết lập miền thời gian rời rạc n trong khoảng từ -5 đến 5 .
- Tạo lập dãy **xung đơn vị (Unit Impulse Sequence)** $\delta[n]$ bằng cách sử dụng biểu thức điều kiện $n == 0$. Hàm `bool2s` đảm bảo tín hiệu có biên độ bằng 1 tại $n = 0$ và bằng 0 tại các vị trí khác.
- Sử dụng hàm `plot2d3` để biểu diễn tín hiệu dưới dạng đồ thị rời rạc.

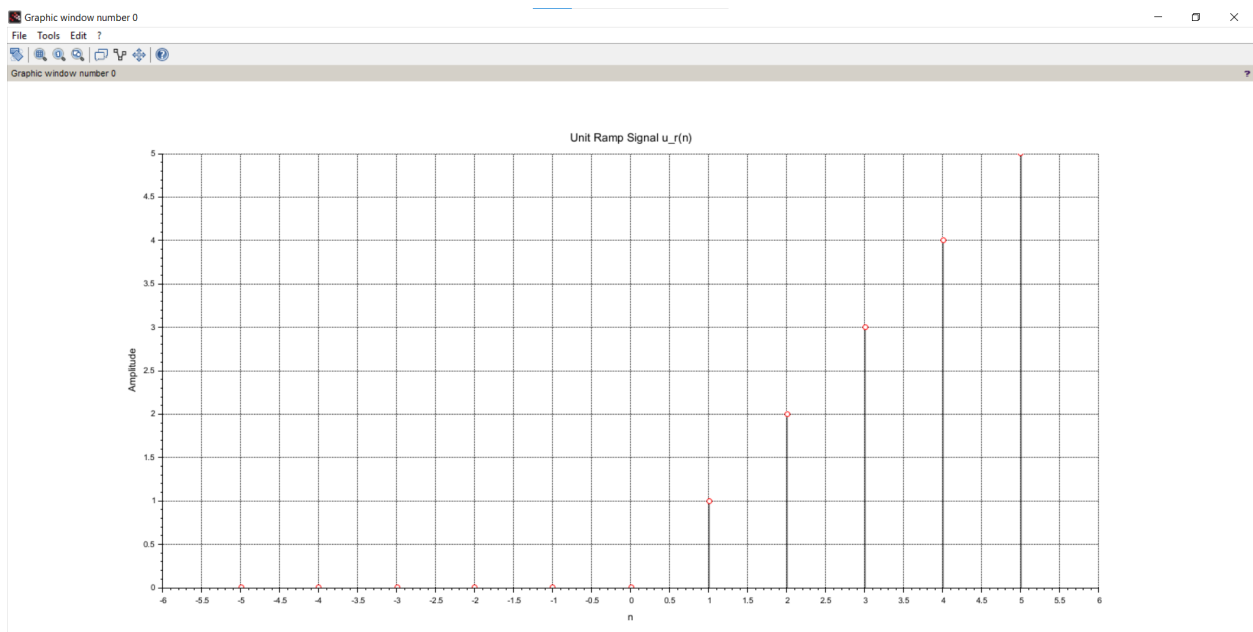
4.2.4 Exercise 4

Sử dụng Scilab để vẽ **tín hiệu dốc đơn vị (unit ramp signal)** $u_r(n)$ với $n = -5 : 5$.

Mã nguồn Scilab cho Exercise 4:

```
1 n = -5 : 5;
2 disp(n);
3 step_s = bool2s(n >= 0);
4 disp(step_s);
5 ramp_s = step_s .* n;
6 disp(ramp_s);
7
8 // ----- Draw -----
9 clf();
10 plot2d3(n, ramp_s);
11 plot(n, ramp_s, "ro"); // Add points to more visible
12 xtitle("Unit Ramp Signal u_r(n)", "n", "Amplitude");
13 xgrid();
14
```

Giải thích: Tín hiệu dốc đơn vị được định nghĩa là $u_r[n] = n$ với $n \geq 0$ và bằng 0 với $n < 0$. Trong đoạn mã trên, ta tạo dãy $u[n]$ thông qua biến **step_s**. Việc nhân n .* **step_s** giúp giữ nguyên giá trị của n khi n dương và đưa biên độ về 0 khi n âm, từ đó thu được dãy dốc đơn vị chính xác theo định nghĩa toán học.



Hình 7: Đồ thị tín hiệu dốc đơn vị (unit ramp signal) $u_r(n)$ trong khoảng $n = -5 : 5$.

4.2.5 Exercise 5

Cho tín hiệu rời rạc $x(n) = \{1, 3\uparrow, -2\}$.

Sử dụng Scilab để vẽ tín hiệu $x(n)$, thành phần tín hiệu lẻ $x_o(n)$ và thành phần tín hiệu chẵn $x_e(n)$. Mỗi tín hiệu được vẽ trên một đồ thị riêng nhưng hiển thị trong cùng một cửa sổ. Sử dụng các hàm `title()`, `xlabel()` và `ylabel()` để chú thích tên cho mỗi đồ thị.

Mã nguồn Scilab cho Exercise 5:

```
1 clear; clc; clf();
2
3 n = -1:1;
4 x = [1, 3, -2];
5 disp(x);
6
7 x_neg = [x(3), x(2), x(1)];
8 disp(x_neg);
9
10 x_e = 0.5 * (x + x_neg);
11 x_o = 0.5 * (x - x_neg);
12 disp(x_e);
13 disp(x_o);
14
```

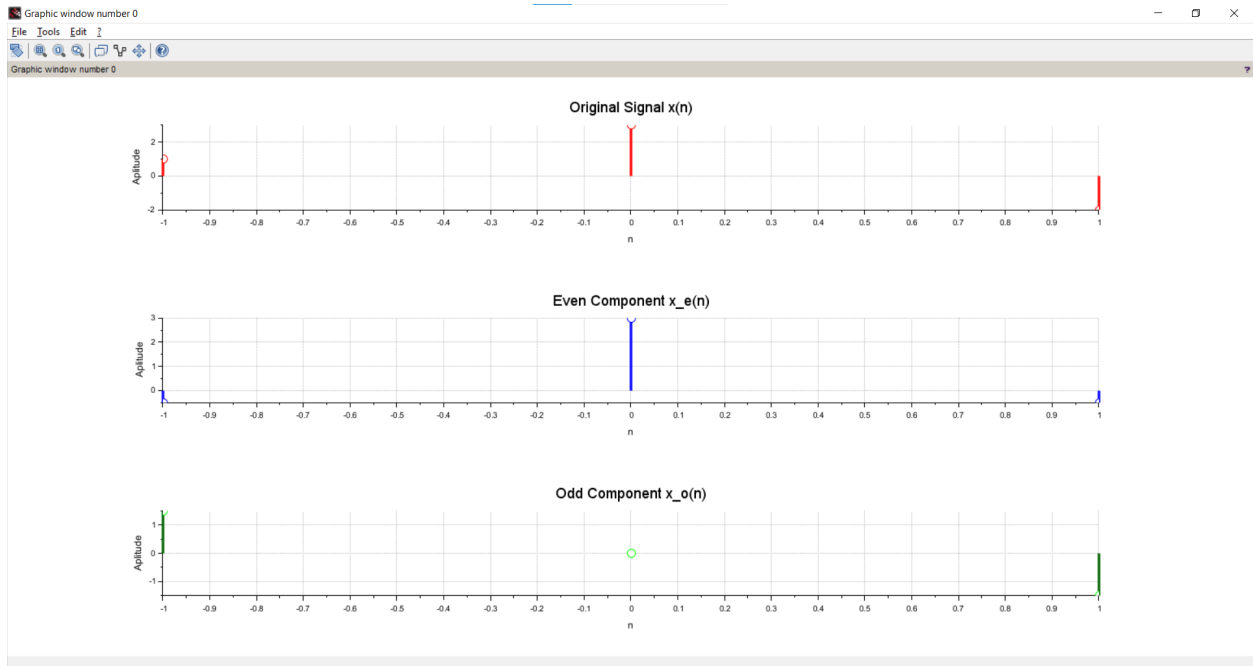
```
15 // --- Configuration for graphic window ---
16 f = gcf();
17 f.background = color('white'); // change background color to white
18
19 // Original Signal
20 subplot(3, 1, 1);
21 plot2d3(n, x);
22 h1 = gce(); // Take handle of previously drawn line
23 h1.children.thickness = 3; // Increase the thickness of line to 3
24 h1.children.foreground = color('red'); // the heavy red
25
26 plot(n, x, "ro");
27 h2 = gce();
28 h2.children.mark_size = 10; // Increase the size of the mark point
29 h2.children.mark_foreground = color('red');
30 title("Original Signal x(n)", "fontsize", 4);
31 xlabel("n"); ylabel("Aplitude");
32 xgrid(color('gray')); // add the gray grid
33
34 // Even Component
35 subplot(3, 1, 2);
36 plot2d3(n, x_e);
37 h3 = gce(); h3.children.thickness = 3;
38 h3.children.foreground = color('blue');
39
40 plot(n, x_e, "bo");
41 h4 = gce(); h4.children.mark_size = 10;
42 title("Even Component x_e(n)", "fontsize", 4);
43 xlabel("n"); ylabel("Aplitude");
44 xgrid(color('gray'));
45
46 // Odd Component
47 subplot(3, 1, 3);
48 plot2d3(n, x_o);
49 h5 = gce(); h5.children.thickness = 3;
50 h5.children.foreground = color('darkgreen');
51
52 plot(n, x_o, "go");
```



```

53 h6 = gce(); h6.children.mark_size = 10;
54 title("Odd Component x_o(n)", "fontsize", 4);
55 xlabel("n"); ylabel("Aplitude");
56 xgrid(color('gray'));
57

```



Hình 8: Kết quả code Scilab vẽ sơ đồ các tín hiệu của Exercise 5.

4.2.6 Exercise 6

Cho hai tín hiệu rời rạc:

$$x_1(n) = \{0\uparrow, 1, 3, -2\} \quad \text{và} \quad x_2(n) = \{0, 1\uparrow, 2, 3\}$$

Xác định tín hiệu $y(n) = x_1(n) + x_2(n)$. Sau đó sử dụng Scilab để vẽ $x_1(n)$, $x_2(n)$ và $y(n)$. Mỗi tín hiệu được vẽ trên một đồ thị riêng nhưng hiển thị trong cùng một cửa sổ. Sử dụng các hàm `title()`, `xlabel()` và `ylabel()` để chú thích cho mỗi đồ thị.

Bài làm

Dựa trên các tín hiệu đã cho, ta tiến hành xác định miền thời gian chung để thực hiện phép cộng:

- $x_1(n)$ có gốc tại 0, miền thời gian ban đầu: $n \in [0, 3]$.

- $x_2(n)$ có gốc tại $n = 0$ (giá trị 1), miền thời gian ban đầu: $n \in [-1, 2]$.
- **Miền thời gian chung:** Xác định bởi $\min(n)$ và $\max(n)$ của cả hai tín hiệu $\Rightarrow n \in [-1, 3]$.

Thực hiện mở rộng các vector tín hiệu bằng cách chèn thêm các giá trị 0 vào các vị trí còn thiếu trên miền thời gian chung:

$$\begin{aligned}x_1(n) &= \{0, \mathbf{0}, 1, 3, -2\} \\x_2(n) &= \{0, \mathbf{1}, 2, 3, 0\} \\y(n) &= x_1(n) + x_2(n) = \{0, \mathbf{1}, 3, 6, -2\}\end{aligned}$$

Kết quả: Tín hiệu tổng $y(n)$ được xác định trên miền $n \in [-1, 3]$ với gốc tọa độ $n = 0$ nằm tại giá trị biên độ bằng 1.

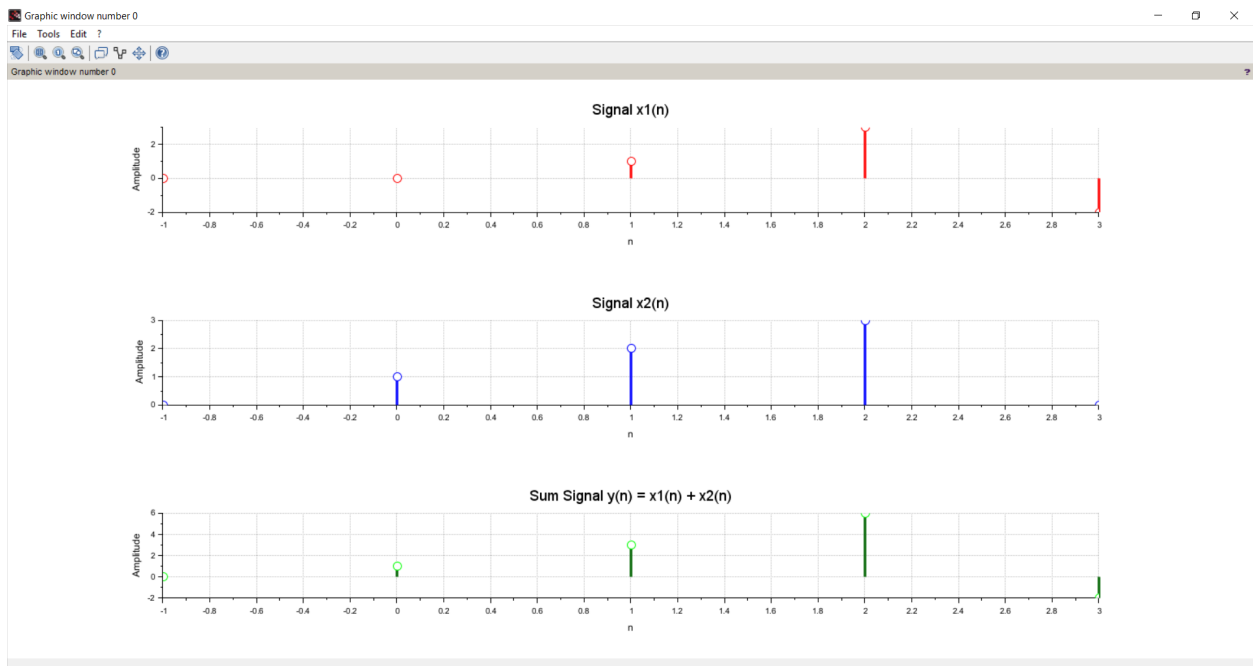
Mã nguồn Scilab cho Exercise 6:

```
1 clear; clc; clf();
2
3 // 1. Định nghĩa miền thời gian chung n = -1 đến 3
4 n = -1:3;
5
6 // 2. Căn chỉnh các vector tín hiệu theo miền thời gian chung
7 x1 = [0, 0, 1, 3, -2]; // x1(n)
8 x2 = [0, 1, 2, 3, 0]; // x2(n)
9 y = x1 + x2;          // y(n)
10
11 // --- Cấu hình của đồ họa ---
12 f = gcf(); f.background = color('white');
13
14 // Vẽ x1(n)
15 subplot(3, 1, 1);
16 plot2d3(n, x1); h1=gce(); h1.children.thickness=3; h1.children.foreground=
    color('red');
17 plot(n, x1, "ro"); h2=gce(); h2.children.mark_size=10;
18 title("Signal x1(n)", "fontsize", 4); xlabel("n"); ylabel("Amplitude");
19 xgrid(color('gray'));
20
21 // Vẽ x2(n)
```

```

22 subplot(3, 1, 2);
23 plot2d3(n, x2); h3=gce(); h3.children.thickness=3; h3.children.foreground=
    color('blue');
24 plot(n, x2, "bo"); h4=gce(); h4.children.mark_size=10;
25 title("Signal x2(n)", "fontsize", 4); xlabel("n"); ylabel("Amplitude");
26 xgrid(color('gray'));
27
28 // Ve y(n)
29 subplot(3, 1, 3);
30 plot2d3(n, y); h5=gce(); h5.children.thickness=3; h5.children.foreground=
    color('darkgreen');
31 plot(n, y, "go"); h6=gce(); h6.children.mark_size=10;
32 title("Sum Signal y(n) = x1(n) + x2(n)", "fontsize", 4); xlabel("n"); ylabel
    ("Amplitude");
33 xgrid(color('gray'));
34

```



Hình 9: Kết quả code Scilab vẽ sơ đồ các tín hiệu của Exercise 6.

4.2.7 Exercise 7

Cho hai tín hiệu rời rạc:

$$x_1(n) = \{0\uparrow, 1, 3, -2\} \quad \text{và} \quad x_2(n) = \{0, 1\uparrow, 2, 3\}$$

Xác định tín hiệu $y(n) = x_1(n) \cdot x_2(n)$. Sau đó sử dụng Scilab để vẽ $x_1(n)$, $x_2(n)$ và $y(n)$. Mỗi tín hiệu được vẽ trên một đồ thị riêng nhưng hiển thị trong cùng một cửa sổ. Sử dụng các hàm `title()`, `xlabel()` và `ylabel()` để chú thích cho mỗi đồ thị.

Bài làm

Tương tự bài tập 6, ta đưa hai tín hiệu về miền thời gian chung $n \in [-1, 3]$:

$$x_1(n) = \{0, \mathbf{0}, 1, 3, -2\}$$

$$x_2(n) = \{0, \mathbf{1}, 2, 3, 0\}$$

$$y(n) = x_1(n) \cdot x_2(n) = \{0, \mathbf{0}, 2, 9, 0\}$$

Nhận xét: Phép nhân tín hiệu trong miền thời gian rời rạc tương ứng với việc điều chế biên độ (Amplitude Modulation). Tại những nơi tín hiệu này bằng 0, nó sẽ làm triệt tiêu tín hiệu kia.

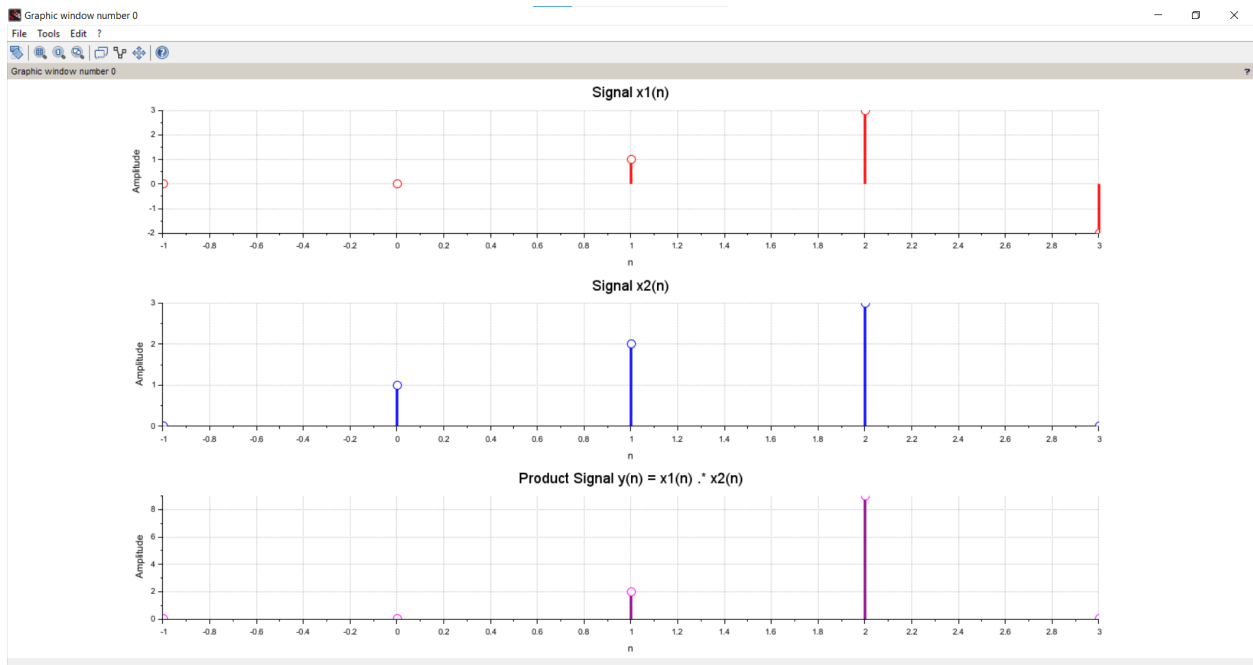
Mã nguồn Scilab cho Exercise 7:

```
1 clear; clc; clf();
2
3 n = -1:3;
4 x1 = [0, 0, 1, 3, -2];
5 x2 = [0, 1, 2, 3, 0];
6 y = x1 .* x2;
7
8 f = gcf(); f.background = color('white');
9
10 // Draw x1(n)
11 subplot(3, 1, 1);
12 plot2d3(n, x1); h1=gce(); h1.children.thickness=3; h1.children.foreground=
    color('red');
13 plot(n, x1, "ro"); h2=gce(); h2.children.mark_size=10;
14 title("Signal x1(n)", "fontsize", 4); xlabel("n"); ylabel("Amplitude");
    xgrid(color('gray'));
```

```

15
16 // Draw x2(n)
17 subplot(3, 1, 2);
18 plot2d3(n, x2); h3=gce(); h3.children.thickness=3; h3.children.foreground=
    color('blue');
19 plot(n, x2, "bo"); h4=gce(); h4.children.mark_size=10;
20 title("Signal x2(n)", "fontsize", 4); xlabel("n"); ylabel("Amplitude");
    xgrid(color('gray'));
21
22 // Draw y(n)
23 subplot(3, 1, 3);
24 plot2d3(n, y); h5=gce(); h5.children.thickness=3; h5.children.foreground=
    color('darkmagenta');
25 plot(n, y, "mo"); h6=gce(); h6.children.mark_size=10;
26 title("Product Signal y(n) = x1(n) .* x2(n)", "fontsize", 4); xlabel("n");
    ylabel("Amplitude"); xgrid(color('gray'));
27

```



Hình 10: Kết quả code Scilab vẽ sơ đồ các tín hiệu của Exercise 7.

4.2.8 Exercise 8

Cho tín hiệu rời rạc $x(n) = \{1, -2, 3, 6\}$. Xác định các tín hiệu sau đây và sử dụng Scilab để vẽ tín hiệu gốc $x(n)$ cùng với tín hiệu đã biến đổi $y_i(n)$. Mỗi cặp đồ thị (gốc và biến đổi) sẽ được hiển thị trong cùng một cửa sổ. Sử dụng các hàm `title()`, `xlabel()` và `ylabel()` để chú thích cho từng đồ thị.

- a. $y_1(n) = x(-n)$
- b. $y_2(n) = x(n + 3)$
- c. $y_3(n) = 2x(-n - 2)$

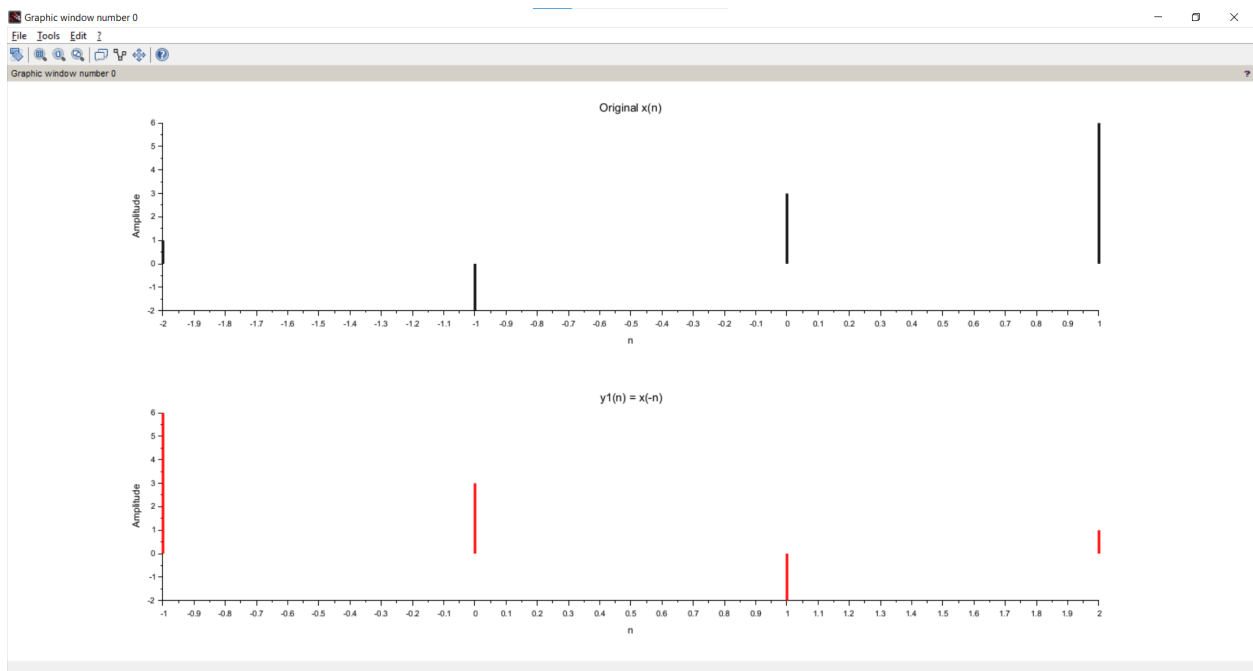
Mã nguồn Scilab cho Exercise 8:

```
1 clear; clc;
2
3 // Original Signal
4 nx = -2:1;
5 x = [1, -2, 3, 6];
6
7 // --- Question a: y1(n) = x(-n) ---
8 ny1 = -1:2;
9 y1 = [6, 3, -2, 1];
10 scf(0); clf(); f=gcf(); f.background=color('white');
11 subplot(2,1,1); plot2d3(nx, x); h=gce(); h.children.thickness=3;
12 title("Original x(n)");
13 xlabel("n"); ylabel("Amplitude");
14
15 subplot(2,1,2); plot2d3(ny1, y1); h=gce(); h.children.thickness=3; h.
    children.foreground=color('red');
16 title("y1(n) = x(-n)");
17 xlabel("n"); ylabel("Amplitude");
18
19 // --- Question b: y2(n) = x(n+3) ---
20 ny2 = -5:-2;
21 y2 = x;
22 scf(1); clf(); f=gcf(); f.background=color('white');
23 subplot(2,1,1); plot2d3(nx, x); h=gce(); h.children.thickness=3;
24 title("Original x(n)");
```

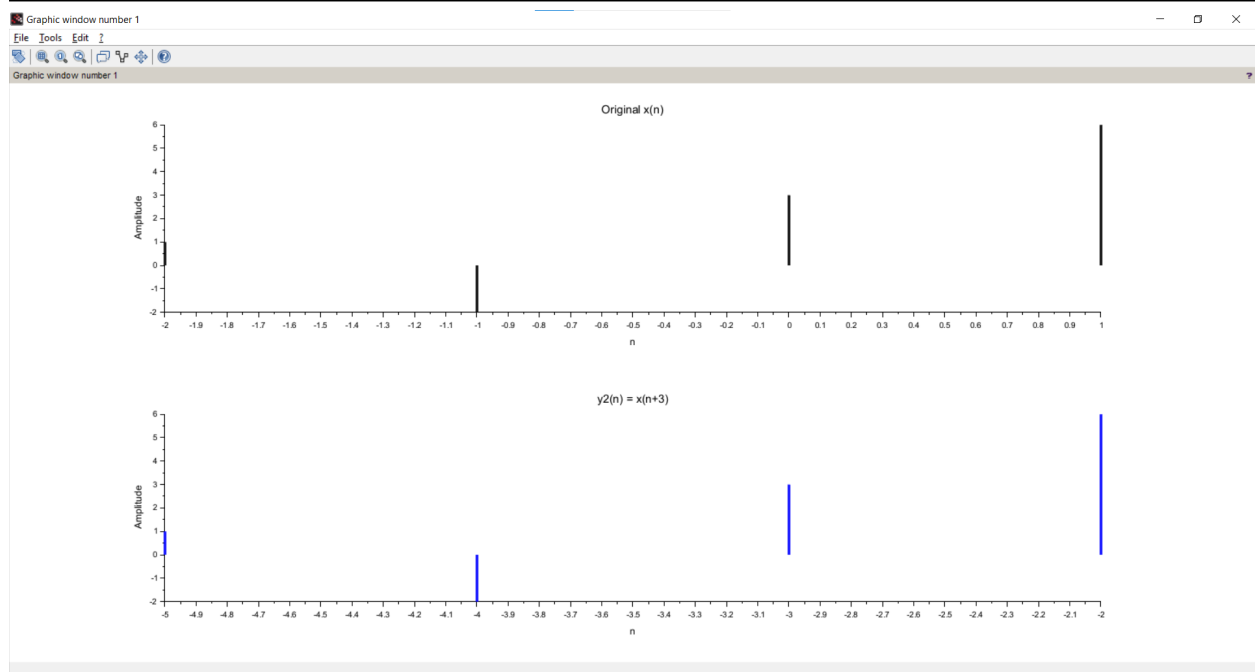
```

25 xlabel("n"); ylabel("Amplitude");
26
27 subplot(2,1,2); plot2d3(ny2, y2); h=gce(); h.children.thickness=3; h.
    children.foreground=color('blue');
28 title("y2(n) = x(n+3)");
29 xlabel("n"); ylabel("Amplitude");
30
31 // --- Question c: y3(n) = 2x(-n-2) ---
32 ny3 = -3:0;
33 y3 = [12, 6, -4, 2];
34 scf(2); clf(); f=gcf(); f.background=color('white');
35 subplot(2,1,1); plot2d3(nx, x); h=gce(); h.children.thickness=3;
36 title("Original x(n)");
37 xlabel("n"); ylabel("Amplitude");
38
39 subplot(2,1,2); plot2d3(ny3, y3); h=gce(); h.children.thickness=3; h.
    children.foreground=color('darkgreen');
40 title("y3(n) = 2x(-n-2)");
41 xlabel("n"); ylabel("Amplitude");
42

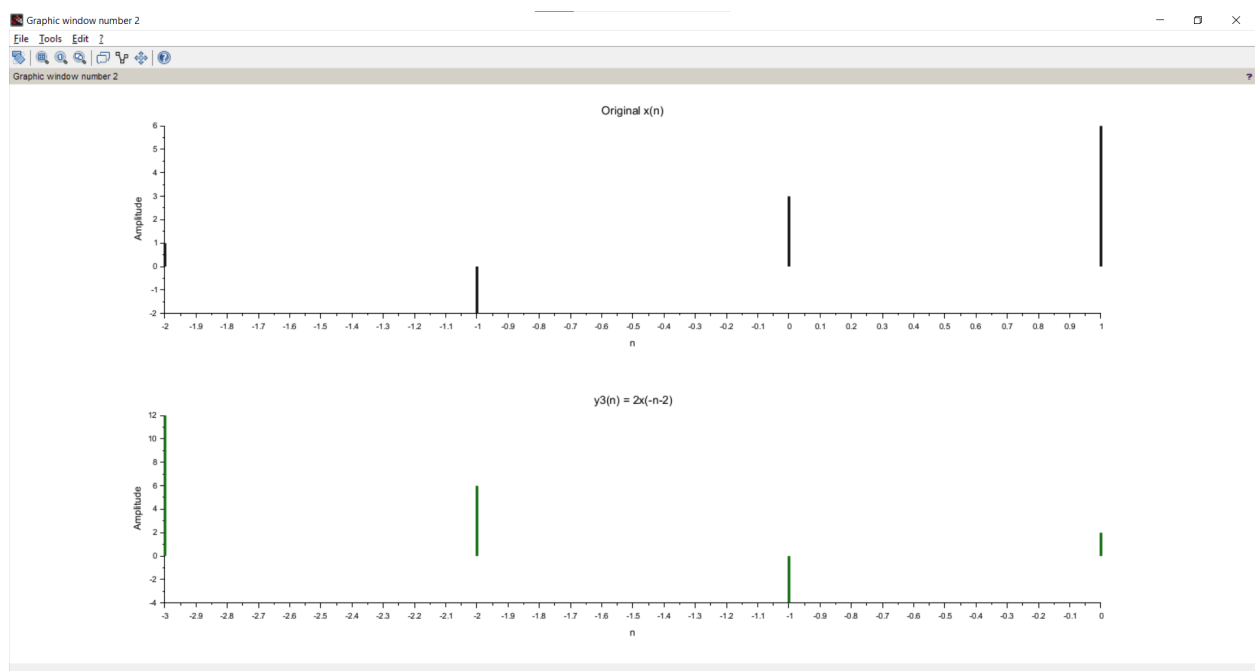
```



Hình 11: Sơ đồ tín hiệu gốc $x(n)$ và tín hiệu chuyển đổi $x(-n)$.



Hình 12: Sơ đồ tín hiệu gốc $x(n)$ và tín hiệu chuyển đổi $x(n + 3)$.



Hình 13: Sơ đồ tín hiệu gốc $x(n)$ và tín hiệu chuyển đổi $2x(-n - 2)$.

5 Bàn luận

5.1 Tổng kết kết quả thực hiện

Thông qua quá trình thực hiện các bài tập trong Lab 1 và Lab 2, các kết quả chính đã đạt được bao gồm:

- **Về mặt kỹ thuật:** Đã làm chủ được môi trường Scilab, vận dụng linh hoạt tư duy *vectorization* để thực hiện các phép toán trên tín hiệu mà không cần sử dụng vòng lặp. Các kỹ năng về trình bày đồ thị (độ dày đường, màu sắc, chú thích) và quản lý cửa sổ đồ họa (*subplot*) đã được thực hiện nhuần nhuyễn.
- **Về mặt kiến thức:**
 - Đã kiểm chứng được quá trình rời rạc hóa tín hiệu tương tự và hiểu rõ ảnh hưởng của tần số lấy mẫu (F_s) cũng như bước lượng tử (Δ) đến chất lượng tín hiệu số.
 - Hiện thực hóa thành công các tín hiệu nền tảng trong DSP như xung đơn vị, bước nhảy và dốc đơn vị.
 - Hiểu sâu sắc về bản chất của các phép biến đổi trong miền thời gian (dịch, đảo, nhân/cộng tín hiệu). Đặc biệt là việc xử lý các tín hiệu có miền thời gian lệch nhau bằng cách cân bằng chiều dài vector.

5.2 Hạn chế và những điểm chưa hoàn thiện

Mặc dù đã hoàn thành đầy đủ các yêu cầu đề bài, bài báo cáo vẫn tồn tại một số điểm cần lưu ý:

- **Xử lý sai số lượng tử:** Trong bài tập 1.2, phép lượng tử hóa hiện tại đang dừng lại ở phương pháp cắt cụt (*truncation*). Trong thực tế, các phương pháp làm tròn (*rounding*) có thể mang lại độ chính xác cao hơn nhưng chưa được phân tích sâu trong bài này.
- **Kỹ năng trình bày báo cáo:** Ban đầu, việc đưa mã nguồn và bảng biểu vào L^AT_EX gặp khó khăn về việc tràn lề (*overflow*) và định dạng bảng tự động nhảy trang. Tuy nhiên, vấn đề này đã được khắc phục bằng cách sử dụng gói lệnh `xltabular`.
- **Đồ họa:** Mặc dù đồ thị đã rõ nét, nhưng việc tự động hóa hoàn toàn các thông số handle đồ họa cho một lượng lớn bài tập vẫn còn tốn thời gian.

5.3 Định hướng phát triển kỹ năng

Để nâng cao năng lực trong môn học Xử lý tín hiệu số, định hướng tiếp theo sẽ tập trung vào:

1. **Nghiên cứu sâu về miền tần số:** Chuyển từ việc quan sát tín hiệu trong miền thời gian sang miền tần số bằng phép biến đổi Fourier (DFT/FFT).
2. **Tối ưu hóa mã nguồn:** Xây dựng các hàm (functions) dùng chung cho các phép biến đổi tín hiệu để tăng tính tái sử dụng của code Scilab thay vì viết script đơn lẻ.
3. **Ứng dụng thực tế:** Tìm hiểu cách áp dụng các bộ lọc (filters) để xử lý nhiễu trên các tín hiệu thực tế được lấy mẫu từ các hệ thống cảm biến.

6 Tài liệu tham khảo

Source code Scilab cho các bài tập: https://github.com/NguyenNgocQuynhNhi/HK25_2_DigitalSignalProcessing_Lab

https://www.scilab.org/sites/default/files/Scilab_beginners.pdf

https://www.overleaf.com/learn/latex/Learn_LaTeX_in_30_minutes

https://www.overleaf.com/learn/latex/List_of_Greek_letters_and_math_symbols